VCFA Import Tool: Technical Design Overview

Oct 20, 2026 · @Yanie

Summary

vcfa-import moves large numbers of vCenter VMs (designed and tested for about 1,800) into VCF Automation namespaces in controlled, resumable waves. It does not move data itself: it prepares and drives VMware's own Mobility Operator, which does the actual import.

Importing by hand means writing one `ImportOperationBatch` manifest per group of VMs. That is workable for a handful of VMs and error-prone at campaign scale. The tool adds the missing campaign layer: inventory, selection, placement, ordering, pacing, failure handling and an audit trail.

It is a single self-contained program (Python standard library only, version 1.1.1) that runs on a jump box as one file or a Windows `.exe`. It is experimental and unsupported, and it is not a VMware or Broadcom product; use is gated behind an accepted disclaimer.

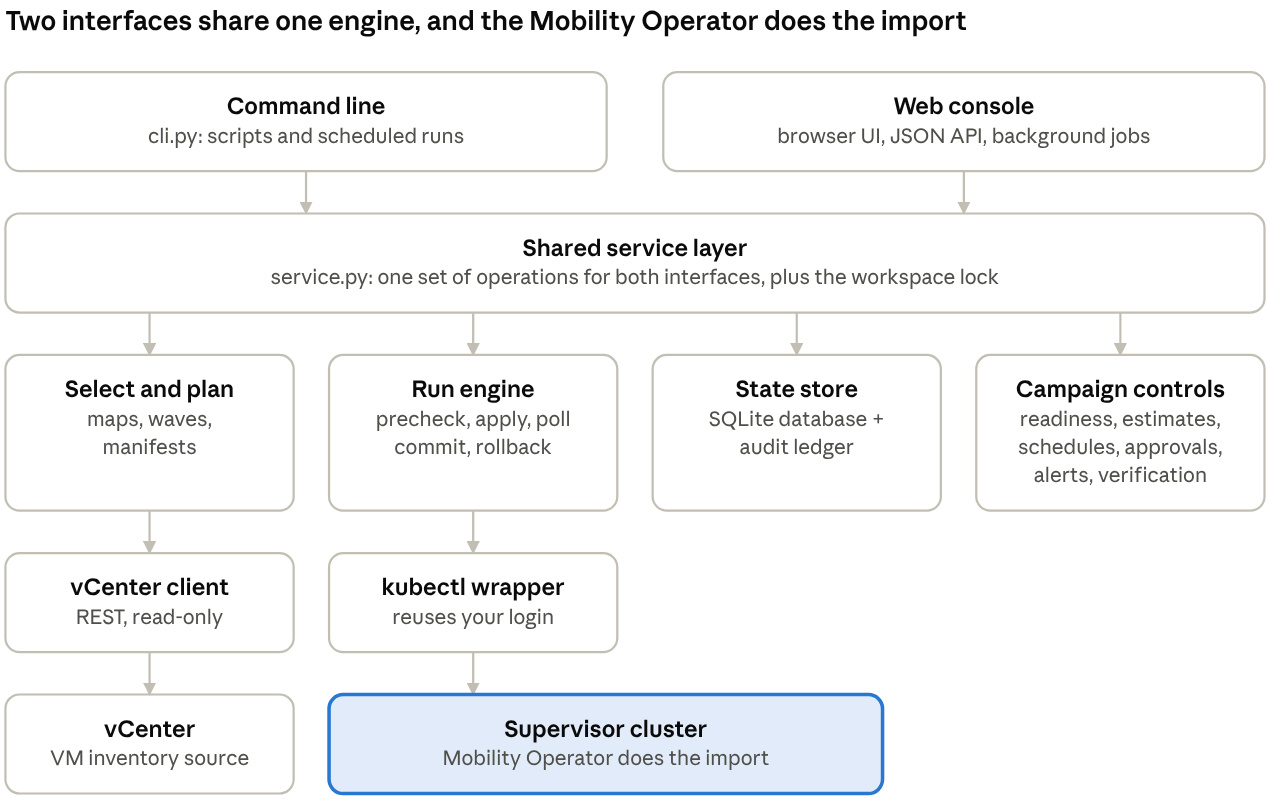
Capabilities

The tool covers the whole life of a migration campaign, from reading vCenter to proving the VMs work afterwards.

- **Discover.** Reads the full VM inventory from vCenter: folders, networks, power state, VMware Tools, tags and disks.
- **Select and place.** Pick VMs by vCenter folder. Folder, network (portgroup) and tag maps decide each VM's target namespace and subnet. A VM that cannot be placed is reported and left out, never guessed.
- **Plan in waves.** Group VMs into ordered waves, keep an application's VMs together, and see an estimated duration per wave.
- **Check first.** Advisory readiness rules flag VMs likely to fail, then the Mobility Operator's own precheck runs before anything is imported.
- **Import with pacing.** Applies batches with capped concurrency, polls the operator until each VM finishes, and holds at an optional commit gate.

- **Recover.** Retry failures, roll back a failed import, skip or abandon VMs. Triage groups failures by cause with the likely fix. A committed import is never touched.
- **Verify.** After commit, checks that each VM is powered on, has Tools running, kept its IP, answers ping and listens on chosen ports.
- **Govern.** Change windows, a two-person approval rule, named user roles (including a read-only viewer link for managers), notifications and an append-only audit ledger.
- **Two interfaces, one engine.** A command line and a browser console share the same engine and database, and can be used interchangeably.
- **Rehearse offline.** A simulated vCenter and operator run a full campaign, including injected failures, with nothing real contacted.

Architecture at a glance



architecture · 4 layers, 2 external systems

The command line and the web console are thin; both call one shared service layer, which uses the same engine and the same database. The tool reads from vCenter and applies batches through `kubectl`; the highlighted Supervisor is where the Mobility Operator actually imports the VMs.

Components

Each row is a Python module (or group of modules) in the `vcfaimport` package. The command line and the web console contain no migration logic of their own; they call the shared service layer.

Layer	Component	Responsibility	Delivers
Interfaces	`cli.py`	Command-line commands: discover, select, stage, precheck, run, status, rollback and more	Scriptable operation, scheduled runs
Interfaces	`web/` (server, API, jobs, UI)	Browser console on a built-in web server; one background job at a time with a live log	Guided five-stage workflow, monitoring, triage
Shared logic	`service.py`	Operations used by both interfaces, plus a workspace lock so only one run touches the cluster	CLI and console always agree
Core	`engine.py`	Preflight, precheck, apply and poll loop, commit gate, retry, rollback, abandon, clean-up	Import with pacing, recovery
Core	`planner.py`, `render.py`	Turn selected VMs into ordered batches; write each batch as a manifest (own YAML writer, no library)	Batched, repeatable manifests
Core	`status.py`	Reads the operator's status and classifies each VM's progress; unknown phases never produce a false verdict	Accurate per-VM state
Core	`state.py`	SQLite database of every VM, batch and transition, plus an append-only ledger	Resume after any interruption, audit trail
Selection	`discovery.py`, `folders.py`, `inventory.py`	Filters, folder/portgroup/tag maps, staging, CSV inventory, an HTML picker	Select and place, plan in waves

Layer	Component	Responsibility	Delivers
Selection	`readiness.py`	Heuristic rules over vCenter facts (power, Tools, CD-ROM, adapters) that grade a VM ready, warn or block	Check first
Adapters	`vcenter.py`	vCenter REST client for inventory, folder tree and NICs; credentials kept in memory only	Discover
Adapters	`kube.py`	Thin wrapper over `kubectl`, so existing Supervisor authentication is reused	Import, no second login path
Campaign controls	`estimate.py`, `schedule.py`	Wave time estimates from measured batches; change windows that never start work that cannot finish in time	Planning, controlled timing
Campaign controls	`access.py`, `settings.py`, `notify.py`, `legal.py`	Users and roles, two-person approvals, guarded settings, Teams/Slack/webhook/email alerts, terms acceptance	Govern
Campaign controls	`verify.py`, `report.py`	Post-import checks on each VM; terminal, CSV and HTML reports	Verify, reporting
Support	`tools/`	Builds the single-file `.pyz` and Windows `.exe`; simulated vCenter and `kubectl` for rehearsal and tests	Offline rehearsal, packaging

How an import runs

A campaign moves left to right through five stages in the console (Discover, Select VMs, Map & Stage, Waves, Execute). The same steps exist as command-line commands.

1. **Discover.** The tool reads every VM, its folder tree and its networks from vCenter. The vCenter password is used once and never stored.

2. **Select.** The person running the campaign ticks vCenter folders (a folder takes its whole subtree) and filters by readiness, tag or application.
3. **Map and stage.** Folder, portgroup and tag maps (plain CSV files) translate each selected VM into a target namespace and subnet. A live preview shows what stages, and which VMs cannot be placed.
4. **Arrange waves.** Folders or VMs are ordered into waves. Each wave shows an estimated duration.
5. **Precheck.** The Mobility Operator tests each VM without moving it. Finished precheck batches are deleted afterwards because they hold storage reservations until removed.
6. **Import.** The person running the campaign reviews the batch plan, then the engine applies batches, one per vCenter folder, within the concurrency limits and polls each one to completion. A commit gate can hold imports until someone approves.
7. **Recover.** Failed VMs are retried, rolled back or skipped. Anything that has already committed is left alone.
8. **Verify and clean up.** Post-import checks run, reports and CSV exports are produced, and batch objects for finished waves are deleted.

The pacing defaults are 10 VMs per batch, 4 batches in flight cluster-wide, 2 per namespace and a 90-minute batch timeout. All are adjustable within guardrails.

Integrations and technology choices

The tool calls vCenter and the Supervisor cluster directly, relies on the Mobility Operator without ever calling it, and can notify people through chat and email.

System	How the tool uses it	Notes
vCenter	REST API, read-only: VM list, folder tree, network adapters, Tools state	Modern `/api` endpoints (vSphere 7.0U2 and later), with a fallback to the older `/rest` ones
Supervisor cluster	Runs `kubectl` to create, read, patch and delete `ImportOperationBatch` objects	Reuses whatever login already works for `kubectl`; needs permission to create `importoperationbatches` in the target namespaces

System	How the tool uses it	Notes
Mobility Operator	Never called directly; the batches the tool applies are what it acts on	VCF 9.1 or later. The API is still `v1alpha3`, so status parsing tolerates unknown fields and phases
Notification services	Teams, Slack, webhook or email, sent in the background	A failed delivery never stops a run

Four choices shape the design:

- **Standard library only.** No packages to install, so the tool ships as one `.pyz` file (Python 3.11 or later) or a Windows `.exe` and is easier to approve onto a jump box. Even the YAML writer is built in.
- **Wrap `kubectl`, don't replace it.** There is no second credential path to maintain or secure.
- **SQLite plus plain files.** State lives in a local database, the audit ledger is an append-only file, and maps are CSVs that a person can read and edit.
- **Self-contained web console.** The interface uses no external fonts, scripts or CDNs because the jump box may be air-gapped.

Safety, security and operations

Because the tool changes production infrastructure, most of its design is about doing less, and doing it traceably.

Protecting the migration

- **Committed imports are final.** Rollback, abandon and clean-up all refuse to touch a VM that has committed. Rollback is only ever requested on a batch that has already run, never at creation.
- **No guessing.** A VM that cannot be mapped to a namespace is left out and reported. An unrecognised operator status keeps a VM "running" rather than declaring success or failure.
- **One run at a time.** Anything that changes the cluster takes a workspace lock; a second CLI run exits and the console answers busy. Two runs on one database were once seen applying duplicate batches.

- **Restartable.** Every state change is written to the database and the audit ledger immediately, so a killed process resumes exactly where it stopped.
- **A precheck expires.** If a VM changes after its precheck (renamed, moved, power or Tools change), it goes back to pending.

Protecting the tool and its secrets

- **Console access.** Every API call needs an access token; on loopback, requests naming another host are refused; destructive actions also need server-side confirmation, and `commitAction: Auto` and campaign reset require typed phrases.
- **Secrets stay out of files.** The vCenter password lives in memory, email passwords are read from an environment variable, and user tokens are stored only as SHA-256 hashes.
- **Terms of use.** Nobody can run the tool until the disclaimer is accepted; acceptance is tied to the exact text, so a new version asks again.

Governance, all optional and off by default

Roles (viewer, operator, admin), a two-person approval rule, change windows and notifications can each be switched on. Approvals are an audit control, not a security boundary. With everything off, the tool behaves as a plain single-operator importer.

Quality evidence

The repository holds about 176 automated tests: seconds for unit tests, roughly two minutes with end-to-end runs against the simulators, and an optional 1,800-VM scale case. A headless-browser stress script drives the real console.

Limits, risks and open questions

The largest risk is that the tool depends on an operator API that is still an early version, and that it is unsupported software.

Risk	Why it matters	How the design responds
Operator API is `v1alpha3` and partly undocumented	Status fields and phases may change or surprise	Status parsing is tolerant; unknown phases keep a VM "running"; findings from a real Supervisor are recorded as tests

Risk	Why it matters	How the design responds
Some operator behaviour is unresolved	Under the commit-wait setting, one build marked a batch complete without a commit; whether rollback works on auto-commit batches is unknown	The tool re-reads a batch before committing and explains rollback errors rather than guessing
Precheck is not a full guarantee	It passed a VM with an installer ISO attached and does not validate subnet references	Readiness rules block known cases; mappings are reviewed in a preview
Leftover precheck batches hold storage	Old ones exceeded a namespace quota and stalled the next import	Finished precheck batches are deleted automatically
Imports cannot be undone once committed	A wrong mapping is permanent	Preview before apply, optional commit gate, approvals and change windows
Experimental and unsupported	No vendor backing; use is at the user's own risk	Disclaimer must be accepted; rehearsal mode lets teams practise offline

Known gaps (backlog)

- Adopting a batch the tool did not create.
- Abandon should also delete a batch's child operations and wait for them.
- Refuse to apply when the cluster already has an import for the same VM, including batches made by hand.
- Warn when two workspaces target the same namespace.
- One job at a time per workspace, by design; a very large campaign proceeds wave by wave.

Open questions for the decision-maker

- Who owns support and upgrades, given the tool's unsupported status and the operator's changing API?

- Is the 1,800-VM scale case, which runs against the simulators, enough, or should a larger rehearsal run on a real Supervisor first?